

CHAPTER 1 NODE JS



Complete Node.js Backend Guide - From Zero to Hero

Let me teach you **everything** about your Node.js backend from scratch! We'll go step by step.

Data Flow Overview (How Everything Connects)

Think of your backend like a **restaurant**:

```
Frontend (Customer) → Backend (Kitchen) → Database (Storage)
    ↓                ↓                ↓
Requests →      Processes →      Stores Data
Data ←      Responses ←      Retrieves Data
```

 Copy

Real Data Flow in Your App:

1. User fills form on website (Frontend)
2. Form data sent to your server (Backend)
3. Server processes & saves to database (MongoDB)
4. Server sends response back to website
5. Website shows success/error to user

 Copy

Project Structure Explained

Your backend is organized like a **building with different floors**:

```
backend/
├── server.js          ← 🏠 MAIN ENTRANCE (App starts here)
├── package.json       ← 📋 SHOPPING LIST (What packages you need)
├── .env               ← 🔑 SECRET KEYS (Passwords, database URLs)
├── src/
│   ├── app.js         ← 🏢 MAIN BUILDING (Express app setup)
│   ├── config/        ← ⚙️ SETTINGS FLOOR
│   │   └── database.js (How to connect to MongoDB)
│   ├── models/        ← 📊 DATA STRUCTURE FLOOR
│   │   └── User.model.js (What a "User" looks like in database)
│   ├── controllers/   ← 🧠 BRAIN FLOOR
│   │   └── auth.controller.js (Logic: register, login, etc.)
│   ├── routes/        ← 🚪 ENTRANCE DOORS
│   │   └── auth.routes.js (URLs: /register, /login, etc.)
```

 Copy

```
| ├── middleware/    ← 🛡️ SECURITY FLOOR
| |   ├── auth.middleware.js (Check if user is logged in)
| |   └── utils/        ← 🛠️ TOOLS FLOOR
| |       └── logger.js  (Record what happens in app)
```

🚀 Let's Understand Each Part

1. server.js - The Starting Point

```
// This is like turning on the lights in your restaurant
require('dotenv').config(); // Load secret keys from .env file

const app = require('./src/app'); // Get the main app
const connectDB = require('./src/config/database'); // Get database connection

const PORT = process.env.PORT || 3000; // Which door number (port) to open

// Connect to database first
connectDB();

// Start the server (open for business!)
app.listen(PORT, () => {
  console.log(`🚀 Server running on port ${PORT}`);
});
```

[Copy](#)

What happens here?

- Loads your secret passwords from `.env`
- Connects to MongoDB database
- Starts listening on port 3000 (like opening your restaurant)

2. .env - Your Secret File

```
# Think of this as your diary - keep it secret!
NODE_ENV=development      # Are we testing or live?
PORT=3000                 # Which door number to use
MONGODB_URI=mongodb+srv://... # Where is your database?
JWT_SECRET=super-secret-key # Password for creating login tokens
```

[Copy](#)

Why is this important?

- Keeps sensitive information separate
- Different settings for testing vs live website
- Never share this file publicly!

3. app.js - The Main Restaurant Setup

```
const express = require('express'); // The framework (like restaurant
equipment)
const cors = require('cors');        // Allow websites to talk to you
const helmet = require('helmet');    // Security guard

const app = express(); // Create the restaurant

// SECURITY (Bouncers at the door)
app.use(helmet()); // Basic security
app.use(cors());   // Allow cross-origin requests

// UNDERSTANDING REQUESTS (Teaching waiter to understand orders)
app.use(express.json()); // Understand JSON data
app.use(express.urlencoded({ extended: true })); // Understand form data

// ROUTES (Menu - what customers can order)
app.use('/api/v1', routes); // All API routes start with /api/v1

module.exports = app; // Give this to server.js
```

 Copy

Think of it as:

- Setting up your restaurant equipment
- Hiring security guards
- Training waiters to understand customer orders
- Creating the menu

4. Models - Data Blueprints

Models define what your data looks like. Like a form template:

```
// User.model.js - What does a "User" look like?
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  name: {
    type: String,      // Must be text
```

 Copy

```

    required: true          // Must provide this
  },
  email: {
    type: String,
    required: true,
    unique: true           // No two users can have same email
  },
  password: {
    type: String,
    required: true,
    minlength: 6          // At least 6 characters
  },
  role: {
    type: String,
    enum: ['user', 'admin'], // Only these two options
    default: 'user'         // If not specified, use 'user'
  }
}, {
  timestamps: true // Automatically add createdAt and updatedAt
});

module.exports = mongoose.model('User', userSchema);

```

What this does:

- Creates a template for user data
- Sets rules (name is required, email must be unique)
- Like a digital form that validates input

5. Controllers - The Brain/Logic

Controllers contain the **actual work** that happens when someone visits a URL:

```

// auth.controller.js - What happens when user registers/logs in?
const User = require('../models/User.model');

// REGISTER - Create new user account
const register = async (req, res) => {
  try {
    // 1. Get data from request
    const { name, email, password } = req.body;

    // 2. Create user in database
    const user = await User.create({
      name,
      email,

```

 Copy

```
        password
    });

    // 3. Send success response
    res.status(201).json({
        success: true,
        data: { user }
    });

} catch (error) {
    // Something went wrong
    res.status(400).json({
        success: false,
        message: error.message
    });
}
};

// LOGIN - Check if user exists and password is correct
const login = async (req, res) => {
    try {
        // 1. Get email and password from request
        const { email, password } = req.body;

        // 2. Find user in database
        const user = await User.findOne({ email });

        // 3. Check if user exists
        if (!user) {
            return res.status(401).json({
                success: false,
                message: 'Invalid credentials'
            });
        }

        // 4. Check if password matches
        const isMatch = await user.matchPassword(password);

        if (!isMatch) {
            return res.status(401).json({
                success: false,
                message: 'Invalid credentials'
            });
        }

        // 5. Create login token
        const token = user.getSignedJwtToken();

        // 6. Send success response
```

```
    res.status(200).json({
      success: true,
      token,
      data: { user }
    });

  } catch (error) {
    res.status(400).json({
      success: false,
      message: error.message
    });
  }
};

module.exports = { register, login };
```

What controllers do:

- Receive requests from users
- Process data (save to database, check passwords, etc.)
- Send responses back to users
- Handle errors gracefully

6. Routes - The Menu/URLs

Routes define which URLs do what:

```
// auth.routes.js - Map URLs to controller functions
const express = require('express');
const { register, login } = require('../controllers/auth.controller');

const router = express.Router();

// When someone visits /register, run the register function
router.post('/register', register);

// When someone visits /login, run the login function
router.post('/login', login);

module.exports = router;
```

 Copy

How this works:

- `POST /api/v1/auth/register` → runs `register` function
- `POST /api/v1/auth/login` → runs `login` function

- Like a restaurant menu: "If customer orders pasta, go to kitchen and make pasta"

7. Middleware - Security Guards

Middleware runs **before** your controller functions:

```
// auth.middleware.js - Check if user is logged in
const jwt = require('jsonwebtoken');
const User = require('../models/User.model');

const protect = async (req, res, next) => {
  let token;

  // 1. Check if token exists in request headers
  if (req.headers.authorization && req.headers.authorization.startsWith('Bearer')) {
    token = req.headers.authorization.split(' ')[1];
  }

  // 2. If no token, reject request
  if (!token) {
    return res.status(401).json({
      success: false,
      message: 'Not authorized'
    });
  }

  try {
    // 3. Verify the token
    const decoded = jwt.verify(token, process.env.JWT_SECRET);

    // 4. Get user from database
    const user = await User.findById(decoded.id);

    // 5. Add user to request object
    req.user = user;

    // 6. Continue to next function
    next();
  } catch (error) {
    res.status(401).json({
      success: false,
      message: 'Token is not valid'
    });
  }
};
```

 Copy

```
module.exports = { protect };
```

How middleware works:

Request → Middleware (check login) → Controller (do work) → Response

 Copy

Complete Request Flow Example

Let's trace what happens when user registers:

Step 1: User submits form

```
// Frontend sends this data
{
  "name": "John Doe",
  "email": "john@example.com",
  "password": "password123"
}
```

 Copy

Step 2: Request hits your server

POST http://localhost:3000/api/v1/auth/register

 Copy

Step 3: Express routes the request

```
// In auth.routes.js
router.post('/register', register); // Calls register function
```

 Copy

Step 4: Controller processes the request

```
// In auth.controller.js
const register = async (req, res) => {
  // Gets { name: "John Doe", email: "john@example.com", password: "password123" }
  const { name, email, password } = req.body;

  // Saves to MongoDB
  const user = await User.create({ name, email, password });
```

 Copy

```
// Sends response back
res.json({ success: true, data: { user } });
};
```

Step 5: Response sent back to frontend

```
{
  "success": true,
  "data": {
    "user": {
      "id": "60f7b3b3b3b3b3b3b3b3b3b3",
      "name": "John Doe",
      "email": "john@example.com",
      "role": "user"
    }
  }
}
```

 Copy

Database Connection Explained

```
// database.js - How to connect to MongoDB
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    // Connect to MongoDB Atlas (cloud database)
    const conn = await mongoose.connect(process.env.MONGODB_URI);

    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error('Database connection failed:', error);
    process.exit(1); // Stop the app if database fails
  }
};

module.exports = connectDB;
```

 Copy

What happens:

- Connects to your MongoDB database in the cloud
- Uses the connection string from your `.env` file

- If connection fails, stops the entire app

Authentication Flow Explained

How Login Works:

1. User sends email + password
2. Server finds user in database
3. Server compares password (using bcrypt)
4. If correct, server creates JWT token
5. Server sends token back to user
6. User stores token (in browser)
7. For future requests, user sends token
8. Server verifies token before allowing access

JWT Token Example:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjYwZjdiM2IzYjNiM2IzYjNiM2IzYjNiMyIsImVudCI6MTYyNjg4MjQxOSwiZXhwIjoxNjI3NDg3MjE5fQ.1x2y3z4a5b6c7d8e9f0g1h2i3j4k5l6m7n8o9p0q1r2s  Copy
```

This token contains:

- User ID
- When it was created
- When it expires
- Signature (proves it's real)

Security Features Explained

1. Password Hashing

```
// Before saving to database
const plainPassword = "password123";
const hashedPassword = "$2b$10$abc123..."; // Encrypted version

// Stored in database: $2b$10$abc123...
// Never store plain passwords!
```

 Copy

2. CORS (Cross-Origin Resource Sharing)

```
app.use(cors()); // Allows frontend (localhost:3000) to talk to backend (localhost:3000)
```

 Copy

3. Rate Limiting

```
// Only allow 100 requests per 15 minutes per IP
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 100
});
```

 Copy

4. Input Validation

```
// In User model
name: {
  type: String,
  required: true,    // Must provide
  minlength: 3,      // At least 3 characters
  maxlength: 50       // Maximum 50 characters
}
```

 Copy

Environment Variables Explained

```
# .env file
NODE_ENV=development    # Tells app if we're testing or live
PORT=3000               # Which port to run on
MONGODB_URI=mongodb+srv://... # Database location
JWT_SECRET=abc123       # Secret key for creating tokens
JWT_EXPIRE=7d           # How long tokens last
```

 Copy

Why use environment variables?

- Keep secrets out of code
- Different settings for different environments
- Easy to change without touching code

Error Handling Explained

Try-Catch Pattern:

```
const register = async (req, res) => {  
  try {  
    // Try to do this  
    const user = await User.create(req.body);  
    res.json({ success: true, data: user });  
  } catch (error) {  
    // If something goes wrong, do this  
    res.status(400).json({  
      success: false,  
      message: error.message  
    });  
  }  
};
```

 Copy

Global Error Handler:

```
// Catches all errors in the app  
app.use((err, req, res, next) => {  
  console.error(err.stack);  
  res.status(500).json({  
    success: false,  
    message: 'Something went wrong!'  
  });  
});
```

 Copy

Key Concepts Summary

1. MVC Pattern (Model-View-Controller)

- **Model** = Database structure (User.model.js)
- **View** = Frontend (React app)
- **Controller** = Business logic (auth.controller.js)

2. RESTful API

- **GET** = Retrieve data
- **POST** = Create new data

- **PUT** = Update existing data
- **DELETE** = Remove data

3. Async/Await

```
// Old way
User.create(data, function(err, user) {
  if (err) console.log(err);
  else console.log(user);
});

// New way (what we use)
try {
  const user = await User.create(data);
  console.log(user);
} catch (error) {
  console.log(error);
}
```

 Copy

4. Promises

```
// Database operations return promises
const promise = User.findOne({ email: 'john@example.com' });
promise.then(user => console.log(user));

// Or use await (cleaner)
const user = await User.findOne({ email: 'john@example.com' });
```

 Copy

What You've Built

Your Node.js backend is a **complete API server** that:

1. **Accepts HTTP requests** from websites/apps
2. **Processes data** (validates, transforms, stores)
3. **Interacts with MongoDB** database
4. **Sends responses** back to clients
5. **Handles authentication** (login/register)
6. **Provides security** (rate limiting, CORS, etc.)
7. **Logs activity** for debugging
8. **Handles errors** gracefully

You now have a **production-ready backend** that can support any frontend application (React, Vue, Angular, mobile apps, etc.)!

This is the foundation that powers modern web applications. Great job building it! 🎉